Comparing Convolutional, Recurrent and Attention-Based Models for Email Spam Classification

Team MVPs: Daniel Alejandro Castillo Martín, Abhi Manishkumar Patel, Eric Ly, Bryan Jacome, Nouroz Ali
Professor Hossein Pourmodheji | November 2, 2026

Summary. We trained a convolutional network (CNN), an LSTM and a Transformer encoder from scratch under one shared protocol and compared them with an order-free baseline that averages word embeddings. On Enron spam, no architecture beat the baseline: its test spam F1 was 0.9900 ± 0.0010, against 0.9888 for the CNN, 0.9859 for the Transformer and 0.9818 for the LSTM. Ablations that removed word-order information from the CNN and the Transformer did not hurt, and a forget-gate initialization change did not help the LSTM. On a second, harder dataset the validation ranking was identical and the LSTM gap to the baseline grew about fourfold. Our reading is that spam in these corpora is mostly lexical: which words appear matters more than how they are arranged.

1. Problem and data

The task is binary email classification into ham (0, legitimate email) and spam (1). The question is whether building structure into a model, through local patterns (CNN), a sequential state (LSTM) or a direct lookup between all words (Transformer), improves spam detection over a baseline that only knows which words occur. The primary metric is spam F1, the F1 score with spam as the positive class; accuracy and macro F1 are also recorded in the notebooks.

The main dataset is the SetFit release of Enron-Spam on Hugging Face (SetFit, n.d.; Metsis et al., 2006): 33,716 emails, 31,716 in the source training partition and 2,000 in the source test partition. Both partitions were audited before any split. Duplicates were detected after normalizing case and whitespace, without changing the text that the models see. From the training partition we removed 51 empty records, 3,137 duplicate rows and 372 emails whose normalized text also appears in the test partition; 19 duplicate rows were removed from the test partition, and no conflicting label groups were found. This leaves 28,156 development emails and 1,981 test emails. The development emails were split 80/20 into training and validation, stratified by label with split seed 42, saved once and loaded by every experiment notebook. The test partition is encoded only inside the final evaluation cell of each notebook.

Table 1. Partitions and class balance for both datasets (the second dataset is described in Section 6).

Dataset	Partition	Emails	Ham	Spam
Enron	Training	22,524	11,681 (51.9%)	10,843 (48.1%)
Enron	Validation	5,632	2,921 (51.9%)	2,711 (48.1%)
Enron	Test	1,981	988 (49.9%)	993 (50.1%)
Kaggle	Training	14,381	7,646 (53.2%)	6,735 (46.8%)
Kaggle	Validation	3,596	1,912 (53.2%)	1,684 (46.8%)
Kaggle	Test	2,000	1,063 (53.1%)	937 (46.9%)

Text is lowercased and split into words and individual punctuation marks. A 10,000-entry vocabulary is built from training text only (minimum frequency 2, ID 0 for padding and 1 for unknown words). Sequences are truncated to 256 tokens and padded within each batch. On Enron the limit truncates 32.2% of training emails and discards 49.6% of training tokens, which keeps each run to minutes but removes the end of long messages.

Shared training protocol

Every model, ablation variant and dataset uses the same protocol: the same splits and vocabulary, cross-entropy loss, AdamW (Loshchilov & Hutter, 2019) with learning rate 0.001 and weight decay 0.01, gradient clipping at norm 1.0, batch size 64 and a budget of 10 epochs. Each configuration is trained with seeds 42, 123 and 2026. For every run we keep the checkpoint from the epoch with the highest validation spam F1, and the test split is scored once, for these frozen checkpoints only; the CNN, Transformer and baseline notebooks refuse to re-run their test cell. Tuning effort is equal and deliberately minimal: no hyperparameter was searched for any model, and layer sizes were chosen, not tuned. The only exploratory run was a five-epoch, single-seed comparison of two LSTM learning rates (validation spam F1 0.9784 at 0.001 against 0.9734 at 0.0003), which agreed with the shared value and was not used further. Parameter counts are similar, from 640,130 (baseline) to 739,586 (LSTM), within 16% of each other.

2. The three models and why

2.1 Convolutional model

Input. Each token ID becomes a learned 64-dimensional embedding, so an email is a row of vectors that the CNN reads from start to end. The padding embedding is zero and padded positions are excluded before pooling.

Architecture. Three parallel one-dimensional convolutions with 100 filters each look at windows of 3, 5 and 7 tokens. Each is followed by ReLU and max-pooling over time, which keeps each filter's strongest match anywhere in the email. The 300 pooled values pass through dropout (0.1) and a linear layer. The model follows the multi-width design of Kim (2014), is written with standard PyTorch layers and has 736,902 parameters: 640,000 in the embeddings, 96,300 in the convolutions and 602 in the classifier.

Why. Convolution slides a small window with shared weights across the input, so a filter that learns a short phrase, such as a call to action or a sales line, detects it at any position. Max-pooling then asks only whether the phrase occurred, which suits spam, where the presence of a cue matters more than its location. Three widths capture phrases of different lengths in a single layer, which also keeps the model close in size to the others. The widths and the number of filters were chosen, not tuned.

2.2 Recurrent model

Input. The same 64-dimensional embeddings are read one token at a time. Sequences are packed, so padding never enters the recurrent state; a check in the notebook confirms that appending extra padding leaves the scores unchanged.

Architecture. A single-layer, unidirectional LSTM with 128 hidden units (Hochreiter & Schmidhuber, 1997). Its final hidden state represents the email and passes through dropout (0.1) and a linear layer. The model has 739,586 parameters: 640,000 in the embeddings, 99,328 in the LSTM and 258 in the classifier.

Why. The LSTM is the course's example of a sequential state: input, forget and output gates control what the cell state keeps, which eases the vanishing gradients of a plain RNN. It is the only one of the three models that preserves word order by construction, and its final hidden state gives a compact summary of the email. One layer keeps its size comparable to the others. The design also carries a known risk that the results bear out: a cue must survive up to 256 state updates to reach the classifier.

2.3 Attention-based model

Input. All 256 positions are presented at once. A token embedding (64 dimensions) is added to a learned position embedding, and a padding mask stops attention and pooling from using padded positions.

Architecture. Two pre-norm encoder layers built from nn.TransformerEncoderLayer (Vaswani et al., 2017), each with 4-head self-attention (16 dimensions per head), a 128-unit GELU feed-forward block, dropout 0.1 and residual connections, followed by a final layer normalization. The outputs are averaged over the real tokens and

a linear layer classifies the email. We wrote the embedding, position, masking, pooling and classifier code; the model has 723,586 parameters, 640,000 of them in the token embedding.

Why. Self-attention lets every word look directly at every other word in one step, so a cue anywhere in the email can be used without passing through a sequential state or being limited to a local window. Attention has no sense of order by itself, which is why positions are added, and they are the component that the ablation removes. Mean pooling gathers evidence spread across the email. Pre-norm is commonly more stable when training from scratch (Xiong et al., 2020); post-norm was not tested. The sizes (64 dimensions, 2 layers, 4 heads) keep the model comparable to the others and were not tuned.

3. Baseline

The baseline maps each token to a learned 64-dimensional embedding, averages the embeddings of the real tokens (masked mean pooling) and applies one linear layer, for 640,130 parameters. It has no convolution, recurrence or attention, so it only knows which words occur, not their order or their neighbours. That makes it the right floor for our question: each of the three models starts from the same kind of embedding and adds one way of building in structure, so any gain over the baseline can be attributed to that structure.

The comparison is fair because the baseline uses the same cleaned splits, vocabulary, 256-token inputs, loss, optimizer settings, 10-epoch budget, seeds and validation spam F1 selection as the other models. Nothing was tuned, which is the same tuning effort as the three models; we preferred equal effort to tuning one model more than another, and Section 8 discusses what this leaves open. Its best epochs were 10, 9 and 10 on Enron. The original Enron baseline checkpoints were not available to the member who ran the final test, so the baseline was retrained on CPU with the unchanged notebook code. All three seeds reproduced the recorded validation spam F1 to six decimals before the test split was scored once.

4. Results

Table 2. Enron spam F1 for every seed. Validation chose each checkpoint; the test split was scored once. Mean ± sample SD over three seeds; best in each column in bold.

Model	Validation spam F1 (selection)				Test spam F1 (scored once)			
	seed 42	seed 123	seed 2026	mean ± SD	seed 42	seed 123	seed 2026	mean ± SD
Baseline	**0.9888**	**0.9882**	**0.9890**	**0.9887 ± 0.0004**	**0.9890**	**0.9910**	**0.9900**	**0.9900 ± 0.0010**
CNN	0.9874	0.9868	0.9871	0.9871 ± 0.0003	0.9889	0.9885	0.9889	0.9888 ± 0.0002
LSTM	0.9834	0.9853	0.9810	0.9832 ± 0.0022	0.9854	0.9820	0.9780	0.9818 ± 0.0037
Transformer	0.9882	0.9851	0.9854	0.9862 ± 0.0017	0.9849	0.9859	0.9868	0.9859 ± 0.0010

The baseline has the highest mean on both splits, 0.9887 ± 0.0004 on validation and 0.9900 ± 0.0010 on test. The CNN is second (0.9871 and 0.9888), the Transformer third (0.9862 and 0.9859) and the LSTM last (0.9832 and 0.9818), with the widest spread across seeds on both splits. In errors, the frozen models misclassify on average 20 (baseline), 22 (CNN), 28 (Transformer) and 36 (LSTM) of the 1,981 test emails. The top three are within 0.0024 of each other on validation, a few test emails, so their order should not be over-read. The LSTM gap is larger and consistent: every LSTM seed is below every baseline seed on both splits.

Why the families differ

The three strongest models share one property: they pool evidence from every position. The baseline and the Transformer average over all real tokens, and the CNN takes the maximum of each filter over the whole email, so a strong spam word or phrase anywhere in the message can decide the prediction. Spam in this corpus appears to be largely lexical, carried by which words occur, and the baseline captures exactly that with the fewest assumptions. The local windows of the CNN and the direct lookup of attention add flexibility, but here they mostly re-learn the same word-presence signal while giving the model more room to fit the training set: two of the three CNN seeds peaked at epoch 6 and scored lower afterwards.

The LSTM is the only model that has to carry its evidence through a sequential state. A cue seen early must survive up to 256 gated updates to reach the final hidden state, and the classifier sees that single summary rather than a pool over positions. This matches what the course taught about recurrent models: the state is a bottleneck, and learning what to keep is harder than detecting that a word occurred. The LSTM was also still improving at the end of the budget (best epochs 10, 10 and 8), so part of its gap may be under-training. This explanation is consistent with the evidence, including the ablations below, but it was not tested directly.

5. Ablations

Each ablation changes exactly one component of one model and keeps the data, seeds, settings and budget fixed. All scores are Enron spam F1, and the differences are variant minus main model.

5.1 Convolutional model: kernel width 1

What changed. Every window width was set to 1, with the same number of filters. Each filter then sees one token at a time, so the model can no longer detect multi-word phrases and behaves like a bag-of-words detector. Smaller kernels also mean fewer weights: 660,102 parameters instead of 736,902.

Table 3. CNN with widths 3, 5 and 7 (full) against widths of 1.

Seed	Validation		Test	
	full	width 1	full	width 1
42	0.9874	0.9890	0.9889	0.9874
123	0.9868	0.9894	0.9885	0.9860
2026	0.9871	0.9891	0.9889	0.9894
Mean ± SD	0.9871 ± 0.0003	0.9892 ± 0.0002	0.9888 ± 0.0002	0.9876 ± 0.0017
Difference (variant minus main)	**+0.0020**		**-0.0012**	

What it shows. On validation, which made every decision, width 1 was higher on all three seeds, by 0.0020 on average, several times the seed SD of the full model (0.0003). On test the full model was higher by 0.0012, about two emails out of 1,981 and smaller than the width-1 seed spread (0.0017), so the test split shows no clear winner. Multi-word windows therefore did not help: with max-pooling, a width-1 CNN only detects whether individual words occur, which is essentially the baseline's signal. The narrower model also trained faster, about 52 seconds per run against 82 on the same GPU, and was cheaper at inference (Section 7).

5.2 Recurrent model: forget-gate bias +1

What changed. The effective initial forget-gate bias was changed from 0 to +1: both LSTM bias vectors are zeroed and +1 is assigned to the forget-gate slice of the input bias only. Architecture, parameter count, data, seeds, learning rate, optimizer and budget are unchanged. A positive forget bias makes the cell retain information early in training (Jozefowicz et al., 2015), so the ablation tests whether retention at initialization helps within our budget. It does not measure how long the model remembers.

Table 4. LSTM with forget-gate bias 0 (main) against +1.

Seed	Validation		Test	
	bias 0	bias +1	bias 0	bias +1
42	0.9834	0.9854	0.9854	0.9828
123	0.9853	0.9835	0.9820	0.9804
2026	0.9810	0.9833	0.9780	0.9839
Mean ± SD	0.9832 ± 0.0022	0.9841 ± 0.0012	0.9818 ± 0.0037	0.9824 ± 0.0018
Difference (variant minus main)	**+0.0008**		**+0.0006**	

What it shows. The +1 variant was better on two of three seeds on validation (mean +0.0008) and on one of three on test (mean +0.0006, about one email), so the effect is within seed noise and its sign is mixed. The spread across seeds roughly halved on both splits (validation SD 0.0022 to 0.0012, test 0.0037 to 0.0018), a hint

of more stable optimization that three seeds cannot establish. Its best epochs were 9, 10 and 7, against 10, 10 and 8. Because this ablation changes optimization at initialization rather than access to order, its null result points to the final-state design, not early forgetting, as the reason the LSTM trails.

5.3 Attention-based model: no positions

What changed. The learned position embedding was removed and nothing else, which removes 16,384 parameters (723,586 to 707,202). With mean pooling, the model then treats each email as a set of words.

Table 5. Transformer with learned positions (full) against no positions.

Seed	Validation		Test	
	full	no positions	full	no positions
42	0.9882	0.9875	0.9849	0.9885
123	0.9851	0.9870	0.9859	0.9860
2026	0.9854	0.9859	0.9868	0.9905
Mean ± SD	0.9862 ± 0.0017	0.9868 ± 0.0008	0.9859 ± 0.0010	0.9883 ± 0.0022
Difference (variant minus main)	**+0.0005**		**+0.0024**	

What it shows. Removing positions did not hurt. On validation the difference (+0.0005) is smaller than the full model's seed SD (0.0017). On test the variant was higher on all three seeds, by 0.0024 on average or about five emails, but seed 123 differs by only 0.0001 and the gap is similar to the variant's own spread (0.0022), so we do not claim that it is better. Attention can find the cue words without knowing where they are.

Together. Two ablations removed a model's access to word order, width-1 kernels and no positions, and neither measurably hurt. Together with the order-free baseline leading both splits, this points to one conclusion for this dataset: the useful signal is which words appear, not how they are arranged. The LSTM ablation tests optimization rather than order, so it does not bear on that conclusion.

6. Second dataset

The second dataset is the Kaggle ham/spam collection (artofintelligence, n.d.): 20,000 English emails after removing missing values and exact duplicates, 10,628 ham (53.1%) and 9,372 spam (46.9%). A stratified test split of 2,000 emails was drawn with seed 42; the shared cleaning then removed 21 normalized duplicates and 2 train-test overlaps, and the rest was split 80/20 into 14,381 training and 3,596 validation emails (Table 1). It has its own 10,000-token vocabulary fitted on its training split, and the 256-token limit truncates 28.1% of its training emails. We chose it because it is the same task with the same input format, so all four models and the protocol apply unchanged, while the source is different and the task is harder: the baseline reaches about 0.964 rather than 0.989, which leaves room for differences below the ceiling that limits Enron. The baseline and the three models were retrained from scratch with the same settings and seeds.

Table 6. Kaggle spam F1 for every seed, same protocol as Table 2; best in each column in bold.

Model	Validation spam F1 (selection)				Test spam F1 (scored once)			
	seed 42	seed 123	seed 2026	mean ± SD	seed 42	seed 123	seed 2026	mean ± SD
Baseline	**0.9656**	**0.9622**	**0.9638**	**0.9639 ± 0.0017**	**0.9628**	**0.9675**	**0.9662**	**0.9655 ± 0.0024**
CNN	0.9622	0.9558	0.9610	0.9597 ± 0.0034	0.9580	0.9512	0.9630	0.9574 ± 0.0059
LSTM	0.9418	0.9392	0.9432	0.9414 ± 0.0020	0.9385	0.9372	0.9464	0.9407 ± 0.0050
Transformer	0.9595	0.9591	0.9571	0.9586 ± 0.0013	0.9585	0.9652	0.9548	0.9595 ± 0.0052

Do the findings hold? Yes, for both main findings. The validation ranking is identical to Enron: baseline (0.9639 ± 0.0017), CNN (0.9597), Transformer (0.9586) and LSTM (0.9414). No architecture beats the order-free baseline, which also leads on test (0.9655 ± 0.0024, about 65 errors out of 2,000). The LSTM is last on both splits again, every LSTM seed is below every baseline seed, and its validation gap to the baseline grows from 0.0054 on Enron to 0.0225 on Kaggle, about four times larger. On test the CNN and the Transformer swap places (0.9574 against 0.9595), a difference of 0.0021 that is smaller than either model's seed spread (0.0059 and 0.0052), so the

middle of the ranking is not resolved. The CNN peaked at the final epoch on all three Kaggle seeds, so a longer budget could change its position.

7. Inference cost

Table 7. Inference cost on Enron validation emails with the seed-42 weights. Peak memory is CUDA peak allocated memory including the weights, except for the LSTM rows (peak above weights and inputs).

Model	Parameters	ms per email (batch 64)	ms per email (batch 1, median)	Peak GPU memory	Hardware
Baseline	640,130	0.036	0.250	23.7 MB	RTX 3060 Laptop
CNN	736,902	0.098	1.630	72.5 MB	GTX 1050 Ti
CNN, width 1	660,102	0.058	1.790	45.3 MB	GTX 1050 Ti
LSTM	739,586	0.120	not measured	43.2 MiB extra	RTX 3060 Laptop
LSTM, bias +1	739,586	0.133	not measured	43.2 MiB extra	RTX 3060 Laptop
Transformer	723,586	0.144	1.313	181.1 MB	Tesla T4
Transformer, no positions	707,202	0.136	1.913	181.9 MB	Tesla T4

Parameter counts compare directly and are within 16% of each other. The baseline, CNN and Transformer were timed with one shared function on 300 validation emails, at batch size 1 after 20 warm-up passes and in batches of 64. The LSTM was timed on one fixed validation batch of 64 (10 warm-up and 50 timed passes), and its parameters take a further 2.82 MiB. All times exclude tokenization, data loading and input transfer. Each owner measured on their own GPU, so the timings are not a ranking across families; even the baseline and the LSTM, which share a GPU, used different procedures. Within a family on one GPU the comparison is valid: width-1 kernels cut the CNN's batched time by 41% and its memory by 38%, removing positions left the Transformer about the same, and the two LSTM variants share one architecture, so their 0.013 ms gap is timing variation. With that caveat, the baseline is also the cheapest model in every measured column.

8. Limitations

Datasets. Both datasets are English email corpora with nearly balanced classes, so the findings may not transfer to other kinds of spam, other languages or strongly imbalanced settings. Enron is close to the ceiling: the baseline reaches 0.990 on test, at the 99% level that the brief warns about, so Enron differences amount to a handful of emails and the Kaggle comparison is the more informative one.

Budget and tuning. Every model had 10 epochs, one configuration and no hyperparameter search. Effort was equal but minimal, so the results describe these configurations rather than each family at its best. The LSTM peaked at the final epoch on two of three Enron seeds and the CNN on all three Kaggle seeds; longer training or tuning could change the order among the neural models, although it would have to close a consistent gap to the baseline.

Seeds. Three seeds give only a rough estimate of the spread, and we ran no significance tests. Gaps below about 0.002 spam F1 are within seed noise, which covers the order of the top three on Enron, every ablation difference on test and the CNN and Transformer swap on Kaggle.

Input length. The 256-token limit drops about half of the training tokens on both datasets. Effects of word order in long emails, beyond the cut, could not be observed, so "order did not help" holds only for this input.

Mechanism and cost. The pooling versus final-state explanation is consistent with the evidence but was not tested; an LSTM that pools over all its states would test it. Inference cost was measured on three GPUs with two procedures, so speed and memory cannot be ranked across families.

Reproducibility. Dataset revisions are recorded as "main" rather than pinned commits, and GPU training is not bit-for-bit reproducible across hardware. Each dataset has a single test split, scored once, so the test results carry the sampling noise of one split.

9. Contributions and references

Table 8. Ownership of the five pieces of the project. Each owner designed, ran and wrote up their piece.

Member	Owned piece	Ablation	Also
Daniel Alejandro Castillo Martín	Shared data preparation; recurrent model (LSTM)	Forget-gate bias +1	Final Enron baseline test and baseline inference cost
Nouroz Ali	Baseline model	Not applicable	Inference cost summary
Bryan Jacome	Convolutional model (CNN)	Kernel width 1	
Abhi Manishkumar Patel	Attention-based model (Transformer)	No positions	
Eric Ly	Second dataset check	Not applicable	Presentation and report preparation

References

artofintelligence. (n.d.). ham-spam [Data set]. Kaggle. https://www.kaggle.com/datasets/artofintelligence/ham-spam

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735-1780. https://doi.org/10.1162/neco.1997.9.8.1735

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95.

Jozefowicz, R., Zaremba, W., & Sutskever, I. (2015). An empirical exploration of recurrent network architectures. Proceedings of the 32nd International Conference on Machine Learning (ICML), 2342-2350.

Kim, Y. (2014). Convolutional neural networks for sentence classification. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), 1746-1751.

Lhoest, Q., et al. (2021). Datasets: A community library for natural language processing. Proceedings of EMNLP 2021: System Demonstrations, 175-184.

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. International Conference on Learning Representations (ICLR).

Metsis, V., Androutsopoulos, I., & Paliouras, G. (2006). Spam filtering with naive Bayes: Which naive Bayes? Third Conference on Email and Anti-Spam (CEAS).

Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32. PyTorch documentation for nn.LSTM, nn.TransformerEncoderLayer and pack_padded_sequence: https://docs.pytorch.org/docs/stable/

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

SetFit. (n.d.). enron_spam [Data set]. Hugging Face. https://huggingface.co/datasets/SetFit/enron_spam

The pandas development team. (2020). pandas-dev/pandas: Pandas [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.3509134

Vaswani, A., et al. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30.

Xiong, R., et al. (2020). On layer normalization in the Transformer architecture. Proceedings of the 37th International Conference on Machine Learning (ICML), 10524-10533.

AI assistance. ChatGPT (OpenAI) was used for the initial notebook starter structure. The Cursor AI agent was used to draft the presentation slides and script, to write the final test and inference cost cells of the baseline notebook, and to edit and format this report. The team reviewed all of this output, and every number in the report was checked against the notebook outputs and result files.